Journal of Systems Architecture 122 (2022) 102358

Contents lists available at ScienceDirect

Journal of Systems Architecture

journal homepage: www.elsevier.com/locate/sysarc

Multi-processor scheduling of elastic applications in compositional real-time systems

Shaik Mohammed Salman *, Alessandro V. Papadopoulos, Saad Mubeen, Thomas Nolte

Mälardalen University, Västerås, Sweden

ARTICLE INFO

Keywords:
Elastic task model
Multi-processors
Real-time scheduling

ABSTRACT

Scheduling of real-time applications modelled according to the periodic and the sporadic task model under hierarchical and compositional real-time systems has been widely studied to provide temporal isolation among independent applications running on shared resources. However, for some real-time applications which are amenable to variation in their timing behaviour, usage of these tasks models can result in pessimistic solutions. The elastic task model addresses this pessimism by allowing the timing requirements of an application's tasks to be specified as a range of values instead of a single value. Although the scheduling of elastic applications on dedicated resources has received considerable attention, there is limited work on scheduling of such applications in hierarchical and compositional settings.

In this paper, we evaluate different earliest deadline first scheduling algorithms to schedule elastic applications in a minimum parallelism supply form reservation on a multiprocessor system. Our evaluation indicates that the proposed approach provides performance comparable to the current state-of-art algorithms for scheduling elastic applications on dedicated processors in terms of schedulability.

1. Introduction

The elastic task model enables convenient modelling of real-time applications that can tolerate some amount of variation in their timing behaviour (changes in interarrival times or changes in execution times) while still maintaining their functional correctness. In uniprocessor systems and multi-processor systems where the resources are fully dedicated to an elastic application, the variation can be managed online by modifying the utilization of the application's tasks to ensure schedulability. Scheduling multiple elastic applications on a shared resource, however, requires the use of reservation-based mechanisms to minimize the impact of such variations on the co-scheduled applications. While different solutions have been proposed for scheduling applications with tasks specified according to the periodic or the sporadic task model under reservation-based mechanisms, there is limited work related to scheduling of elastic applications under reservation schemes. In this context, we propose a scheduling framework for executing elastic applications under the minimum-parallelism periodic resource supply model (MPS) on multi-processor systems. While the elastic task model allows defining the task parameters to capture the variable timing requirements of the application, the minimum-parallelism periodic resource supply model [1,2] provides a relatively simpler mechanism to reserve the resource supply on a multi-processor system. Moreover, under certain period assignment constraints, the MPS model dominates other comparable state-of-art techniques [2].

Concretely, in this paper, we address the following questions:

- Q1 Given an application with elastic tasks, what is a feasible bandwidth required to schedule the applications in an MPS reservation on a multiprocessor system?
- Q2 Given a fixed bandwidth MPS reservation, what mechanism can the elastic application tasks adopt to remain schedulable?
- Q3 Given an elastic application, can a schedulable reservation be found if the application requests for a modified bandwidth reservation?
- Q4 What is the trade-off between the proposed approach and the current state-of-art approaches?

2. Proposed system model

We consider the scheduling of a real-time application specified according to the elastic task model with each task having an implicit deadline (See Section. 2.1). At the system level, we assume that the CPU resources are made available to each application according to the multi-processor Minimum Parallelism Supply Model (MPS) [2,1] (see Section 2.2). We assume that each application provides its own

* Corresponding author.
E-mail addresses: shaik.mohammed.salman@mdh.se (S.M. Salman), alessandro.papadopoulos@mdh.se (A.V. Papadopoulos), saad.mubeen@mdh.se (S. Mubeen), thomas.nolte@mdh.se (T. Nolte).

https://doi.org/10.1016/j.sysarc.2021.102358
Received 26 October 2021; Received in revised form 15 November 2021; Accepted 17 November 2021
Available online 2 December 2021
1383-7621/© 2021 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

Table 1
An elastic task set.

Task ID	WCET	T_i^{min}	T_i^d	T_i^{max}	E_i
τ_1	4	40	120	240	1
τ_2	8	40	80	360	0.75
τ_3	12	240	240	480	0.5
τ_4	12	200	240	600	0.25
τ_5	4	40	40	40	0

local scheduling mechanism, based on the partitioned dynamic-priority scheduling implementing the Earliest Deadline First (EDF) policy. The dedicated full processors do not need a system-level scheduler but rather use the application provided scheduler. The partial processor can be managed by any scheduler which can ensure that the partial processor provides supply according to the periodic resource model [3].

2.1. The basic elastic task model

We define an elastic application **A** as a set of n tasks where each task is specified as $\tau_i = \{C_i, T_i^{min}, T_i^d, T_i^{max}, E_i\}$. Here, C_i is the Worst-Case Execution Time (WCET) of the task while T_i^{min} and T_i^{max} specify the minimum and the maximum interarrival time between consecutive jobs of τ_i. T_i^d represents the desired period of the task τ_i. The elastic co-efficient E_i determines the flexibility of the task τ_i to change in its period [4]. For instance, if E_i is defined to be in the range $[0, 1]$, $E_i = 0$ ensures that the task's desired period cannot be modified, implying that $T_i^{min} = T_i^d = T_i^{max}$. Similarly, $E_i = 1$ indicates that the task's period can be modified to take up values up to its maximum period. Further, the elastic coefficient acts as a weighting factor, determining the extent to which its utilization can be reduced in relation to other tasks. We use T_i (without any postscript) to indicate the current inter-arrival time of τ_i. An example of an elastic taskset is shown in Table 1. Here, the period of the task τ_1 can be extended up to its maximum period, while the task τ_5 can only execute with its desired period.

The utilization of a task τ_i is defined as $U_i = \frac{C_i}{T_i}$. The minimum and maximum utilization of each task is represented by $U_i^{min} = \frac{C_i}{T_i^{max}}$ and $U_i^{max} = \frac{C_i}{T_i^{min}}$, respectively. The desired utilization is represented by $U_i^d = \frac{C_i}{T_i^d}$. The desired application utilization is given by the sum of the desired utilization of the individual tasks. i.e., $U^d = \sum_{i=1}^{n} U_i^d$. Similarly, the minimum and maximum application utilization is given by $U^{min} = \sum_{i=1}^{n} U_i^{min}$ and $U^{max} = \sum_{i=1}^{n} U_i^{max}$. The relative deadline of each job of an elastic task is equal to its current period at runtime. If a task requests for a change in its current period, its desired utilization U_i^d is updated and a schedulable utilization for rest of the tasks is calculated. An elastic application is said to be schedulable if $U^d \leq U^{ub}$, where U^{ub} is the utilization upper-bound for a given scheduling algorithm. It is assumed that the deadline is elastic-implicit. Note that when the utilization of a task is changed to accommodate increased utilization of another task, the reduced utilization is used to check for schedulability instead of the initial desired period.

2.2. Minimum-parallelism resource supply model

The resource supply provided to the application is based on the minimum parallelism resource supply model [1,2]. Here the resource supply is provided by reserving m dedicated processors for an application and at most one periodic resource model based partial processor [3] for the exclusive use of the application. Here, the partial processor is defined as $\Gamma(\Theta, \Pi)$, where Θ is the periodic resource allocation time and Π is the resource period. Essentially, the periodic resource Γ provides an application with Θ time units of CPU time every Π time units. The utilization of the partial resource supply is defined as $U_\Gamma = \frac{\Theta}{\Pi}$.

Table 2
Taskset with desired utilization.

Task ID	U_i^d
τ_1	0.55
τ_2	0.50
τ_3	0.50
τ_4	0.45
τ_5	0.25
τ_6	0.15

Fig. 1 shows the resource supply provided according to the MPS model with 3 processors. Here, processor 1 and processor 2 are fully dedicated to the application, while processor 3 is available partially and modelled according to the periodic resource model. In the worst case, i.e., when the resource budget is exhausted at the beginning of its period and the next instant it is available close to the end of the next period, the no supply interval of the partial processor is equal to $2(\Pi - \Theta)$ [3].

Generating the resource supply reservation parameters. As the MPS model requires reserving m full processors and one partial processor, we need to identify the number of dedicated processors along with the resource supply parameters of the partial processor necessary to schedule the application. We identify these values based on the initial desired utilization. As we consider partitioned EDF scheduling, we apply the reasonable allocation decreasing (RAD) partitioning heuristic specified in [5]. For example, consider the taskset with its desired utilization in Table 2. The tasks τ_1 and τ_4 are assigned to the same processor since their utilization sums up to one. Similarly τ_2 and τ_3 are assigned to another processor. As τ_5 and τ_6 have their utilization much less than the utilization limit for a fully dedicated processor under EDF, they are assigned a partial processor. We apply the method described in [3] to identify the capacity and the period of the partial processor to schedule τ_5 and τ_6.

Schedulability bounds. For the dedicated processors, we assume the uniprocessor utilization bound of one for partitioned EDF. For the partial processor, we use the utilization bound defined according to the periodic resource model. Particularly, the application tasks assigned to the partial processor with utilization $U_{\mathbf{A}}$ are schedulable under EDF scheduling policy if the partial processor resource supply utilization satisfies Eq. (1) (Eq. 21 in [3]).

$$U_{\Gamma,EDF}(k) = \frac{(k+2) \cdot U_{\mathbf{A}}}{k + 2(U_{\mathbf{A}})}, \tag{1}$$

where,

$$k = \max \left\{ k \in \mathbb{Z} \middle| (k+1)\Pi - \Theta - \frac{k\Theta}{k+2} < T^{min} \right\} \tag{2}$$

and T^{min} is the smallest period among the tasks assigned to the partial processor.

3. Proposed solution

As the minimum parallelism resource supply model allows at most one partial processor, the remaining resources are provided by dedicated processors. Therefore, we first consider scheduling of elastic applications on dedicated multiprocessors. In this section, we summarize the current state-of-art solution to the scheduling problem based on the partitioned heuristics proposed by Orr et al. [6]. We then extend this solution to include the partial processor.

3.1. Scheduling elastic applications on dedicated multi-processors

The schedulability of elastic applications on dedicated multiprocessors was evaluated by Orr et al. [6] using both global as well

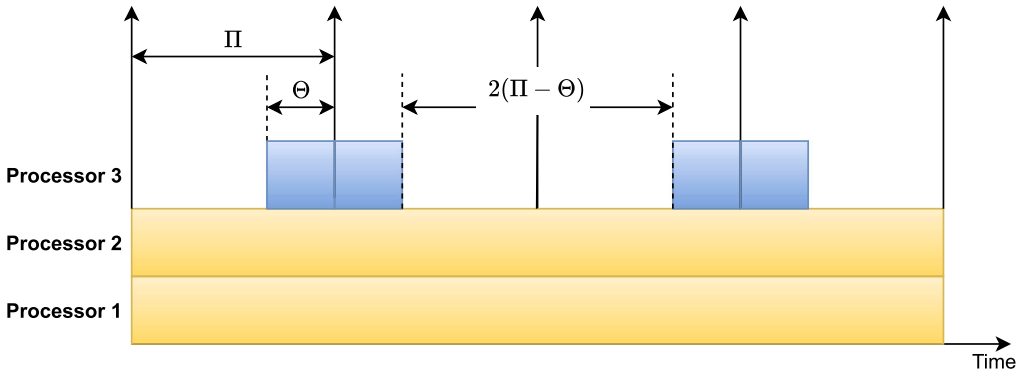


Fig. 1. Minimum parallelism resource supply model.

as partitioned scheduling approaches. Their work highlighted the relatively better schedulability performance of the partitioned approach when compared to global scheduling for elastic applications. We summarize their approach below and refer to it as the global re-partitioning approach in the rest of the paper.

The global re-partitioning approach. This approach involves iteratively compressing the utilization and re-partitioning the taskset until a schedulable partition is found. The algorithm iterates over the values in the range $[0, \Phi]$, where Φ is calculated according to (4), to find the smallest value of λ such that the utilization assigned to each task according to (3) can result in a schedulable partition. During each iteration, it assigns a value to λ and calculates the utilization to be assigned to each task. For this set of utilization values, it then applies a predetermined partitioning heuristic such as the "reasonable allocation decreasing (RAD)" [5], where the tasks are ordered according to monotonically decreasing utilization values and each task is then assigned to the processor on which it fits. The task to processor assignment can be based on

1. First Fit: Assign the task to the first processor on which it fits. i.e., the task meets its schedulability conditions.
2. Best Fit: Assign the task to the processor with minimum remaining capacity after its allocation.
3. Worst Fit: Assign the task to the processor with maximum remaining capacity after its allocation.

If a task remains unassigned, the value of λ is incremented by a granularity constant ϵ and the process is repeated until either all tasks are assigned to processors, resulting in a successful allocation, or the value of λ exceeds Φ, resulting in failure.

$$U_i = \max(U_i^d - \lambda E_i, U_i^{min}) \tag{3}$$

$$\Phi = \max\left(\frac{U_i^d - U_i^{min}}{E_i}\right) \tag{4}$$

An example. Consider an elastic taskset with values given in Table 3 that needs to be scheduled on a 2 core processor. According to the first fit heuristic and considering the desired utilization values, the tasks τ_1 and τ_2 are assigned to core 1 while τ_3 and τ_4 are assigned to core 2. At runtime, suppose that the task τ_1 requests for a new utilization equal to 0.65. The algorithm tries to find a suitable value for λ such that a schedulable partition is found. Observing the utilization values in this example, one can notice that the new utilization request of the task τ_1 could have been easily accommodated by changing the utilization of task τ_2 from its desired value to its minimum utilization value without the re-partitioning step. Based on this observation, we propose a new algorithm that applies the utilization modification algorithm on a per-core basis and only re-partitions the taskset if the per-core approach fails to successfully accommodate the requests for increased utilization values.

Table 3
An elastic task set.

Task ID	U_i^{min}	U_i^d	U_i^{max}	E_i
τ_1	0.25	0.55	0.75	1
τ_2	0.25	0.45	0.65	1
τ_3	0.25	0.50	0.75	1
τ_4	0.25	0.50	0.75	1

Table 4
An elastic task set with unschedulable demand.

Task ID	U_i^{min}	U_i^d	U_i^{max}	E_i
τ_1	0.25	0.55	0.75	1
τ_2	0.35	0.45	0.65	1
τ_3	0.15	0.50	0.75	1
τ_4	0.15	0.50	0.75	1

Utilization modification on a single core. While the re-partitioning approach is relatively straightforward to implement, one disadvantage of this approach is that if a taskset is not schedulable, the global re-partitioning can cause multiple task migrations resulting in additional book-keeping overhead. Further, if the temporal isolation strategies such as cache partitioning are part of an overall solution, especially those based on task-based cache allocation, then task migrations may require re-partitioning of the caches adding to the overhead. One way to minimize such overheads is to limit the utilization modification to tasks running on the same processor or the partial processor as the task requesting the increased utilization.

An example. While the request for increased utilization of the task τ_1 in the previous example could be successfully managed following the per-core utilization approach, this approach can fail to find a schedulable solution for other requests and tasksets. To illustrate this, consider the elastic taskset as given in Table 4, which is similar to the previous example but with modified minimum utilization values. Based on the desired utilization values, the tasks τ_1 and τ_2 are assigned to core 1 while τ_3 and τ_4 are assigned to core 2. At runtime, suppose that the task τ_1 requests for a new utilization equal to 0.75. Applying the per-core approach would require the utilization of the task τ_2 to be reduced to 0.25. Since the minimum utilization of task τ_2 is greater than the required value, this results in an unschedulable condition. If re-partitioning was applied, however, the request could be successfully handled by assigning τ_1 and τ_3 to the same core and reducing the utilization τ_3 to 0.25. Similarly, τ_2 and τ_4 could be assigned to the same core to ensure schedulability.

Based on these observations, we describe next a combined approach that exploits the advantages offered by the per-core approach while improving the range of schedulable elastic tasksets through limited re-partitioning.

The combined approach. Although the original re-partitioning approach finds schedulable solutions for most of the evaluated scenarios (See Section 4), the associated overheads can be minimized by first applying the utilization modification algorithm to the tasks running on the same processor as the task requesting for increased utilization to find a schedulable utilization on that processor. Since the complexity of the utilization modification algorithm is a function of the number of tasks and the number of processors, by reducing the number of tasks whose utilization should be adjusted, the efficiency of the algorithm can be improved. Indeed, if a schedulable utilization is not found, then re-partitioning considering all the cores and the complete taskset cannot be avoided and as such, the overheads related to task migration (and if applicable, cache partitioning) remain the same as in the original approach. The general steps for the combined approach are as follows:

1. Order the taskset either in an (i) arbitrary manner, or (ii) monotonically increasing utilization, or (iii) monotonically decreasing utilization.
2. Order the processors in some arbitrary manner.
3. Allocate tasks to processors according to a "Reasonable Allocation" scheme.
4. When a task makes a request for increased utilization, apply the iterative utilization modification algorithm only to tasks on the same core.
5. If such a request fails, apply the iterative utilization modification algorithm including re-partitioning by considering all the cores.

The combined approach is able to find schedulable solutions similar to the original re-partitioning approach but with improved efficiency since the re-partitioning step is applied only if the per-core utilization modifications fail. In some of the evaluated scenarios, the results indicate per-core utilization modifications are sufficient to keep the application schedulable while completely avoiding the re-partitioning step. In limited cases where the per-core utilization fails, the re-partitioning approach successfully finds a schedulable partition.

3.2. Extension to minimum parallelism resource supply model

The approaches discussed in the previous section considering dedicated processors can be extended to the minimum parallelism resource supply reservation model with some minor modifications. One of the key distinguishing characteristics between the MPS reservation model and the fully dedicated processors is the presence of the partial processor. Since the partial processor is only available periodically, under worst-case conditions, there can be a no supply interval equal to $2(\Pi - \Theta)$ (See Fig. 1). Any task with a period less than this no supply interval cannot be allocated to the partial processor. As a consequence, the re-partitioning step should not only consider the utilization but also the period of the tasks. Moreover, in the case where both per-core utilization modification, as well as re-partitioning, fail due to the period constraints, depending on the availability of resources on the core executing the partial processor, a re-allocation of the bandwidth on the partial processor can be considered. The general steps of the proposed approach are as follows:

1. Order the taskset (i) arbitrary manner (ii) monotonically increasing desired utilization or (iii) monotonically decreasing desired utilization.
2. Order the processors in some arbitrary manner.
3. Allocate tasks to processors according to a "Reasonable Allocation" scheme.
4. if any task remains unallocated, generate resource supply parameters for the partial processor according to the initial desired utilization values of the unallocated tasks.
5. When a task makes a request for increased utilization, apply the iterative utilization modification algorithm only to tasks on the same core (or partial processor as applicable).
6. if such a request fails, apply the iterative utilization modification algorithm by including re-partitioning considering all the dedicated cores. Include partial processor only if the failing request belongs to the partial processor.
7. During re-partitioning, only allocate tasks whose periods are greater than the worst-case no-supply duration on the partial processor while ensuring that the total utilization of the task assigned to the partial processor remains below the partial processor utilization according to Eq. (1).
8. If this request fails, modify the bandwidth of the partial processor if resources are available and repeat steps 3 and 4.

Table 5
Different configuration settings.

Configuration	Maxmin	No. of tasks	No. of processors
1	1, 0.5	100	2
2	1, 0.5	100	4
3	1, 0.5	100	8
4	1, 0.5	100	16
5	1, 0.5	200	2
6	1, 0.5	200	4
7	1, 0.5	200	8
8	1, 0.5	200	16
9	0.5, 0.5	100	2
10	0.5, 0.5	100	4
11	0.5, 0.5	100	8
12	0.5, 0.5	100	16
13	0.5, 0.5	200	2
14	0.5, 0.5	200	4
15	0.5, 0.5	200	8
16	0.5, 0.5	200	16

4. Evaluation

We evaluated the proposed approach for the case of fully dedicated processors by considering randomly generated tasksets with a varying number of tasks and utilization values along with a different number of processors and partitioning heuristics. Here we present the results of the 16 such configurations as detailed in Table 5. For each configuration, we measured the number of successful requests along with the computation time (based on the Query Performance Counter provided by the Windows OS) for a prototype implementation of the proposed algorithms.

Taskset generation. The desired utilization of each task of a taskset was generated using the algorithm proposed by Griffin et al. [7]. The minimum utilization was generated by subtracting a randomly generated percentage (taken from a uniform distribution) from the desired utilization. Similarly, the maximum utilization was generated by adding a randomly generated percentage to the desired utilization while ensuring that the maximum utilization for each task was under one. Each request for increased utilization was generated by selecting a random task and a uniformly distributed random utilization value from within the task's desired and maximum utilization. The elastic coefficient was assigned to each task by randomly generating real values taken from a uniform distribution between [1–10].

Configurations. To compare the performance of the proposed approaches with a state-of-art method, we evaluated different configurations by varying the number of processors, and tasks along with different percentage limits on the utilization values to define the maximum and minimum utilization of each task. For the desired utilization, the maximum utilization of each task was set to be not greater than 0.5. Further, the tasks were ordered according to monotonically decreasing utilization values. The first fit approach was used as the task to processor assignment strategy. Table 5 shows the different configurations where the column "maxmin" indicates the maximum and minimum values used to define the utilization values. For example, the values

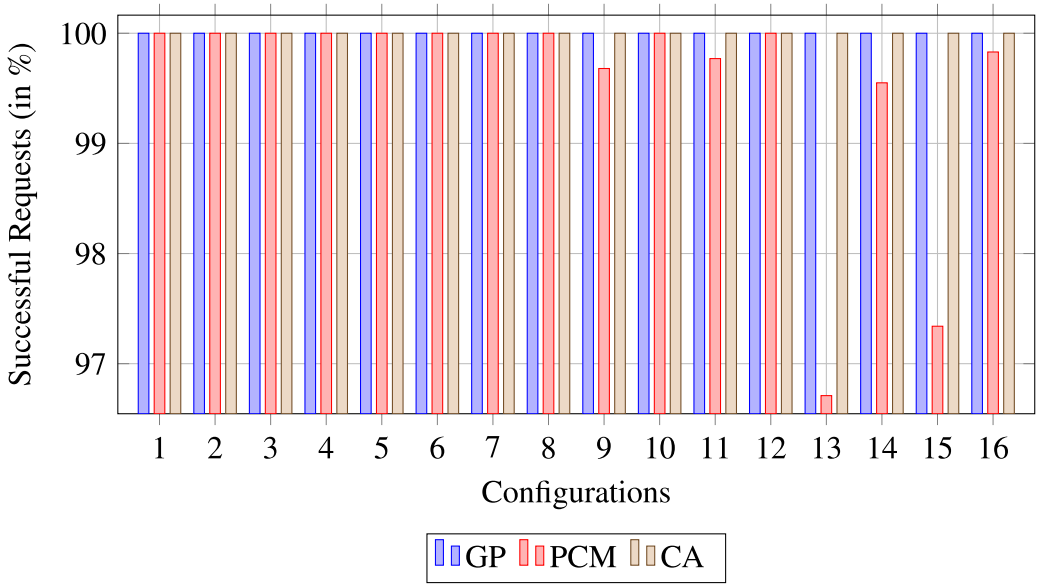


Fig. 2. Schedulability performance of the different approaches.

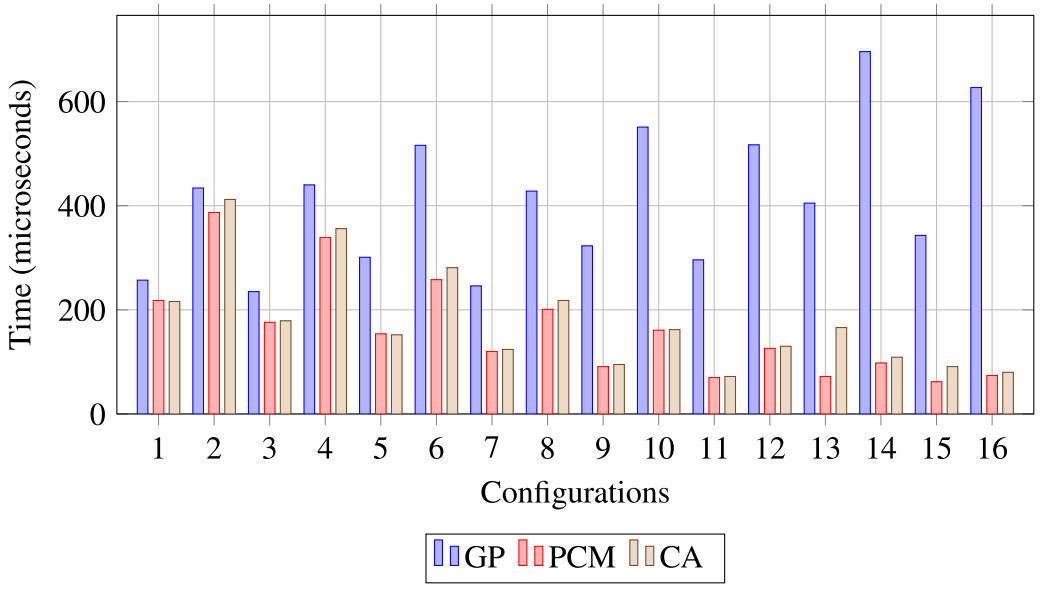


Fig. 3. Average computation times of the different approaches.

$(1, 0.5)$ indicate that the maximum utilization that a task can have is in the interval $[U_d, U_d + (1 \cdot U_d)]$, while the minimum utilization a task can have is in the interval $[U_d - (0.5 \cdot U_d), U_d]$. The values under the column "configuration" refer to the respective (row) configurations settings and are used as identifiers on the x-axis in the associated graphs.

4.1. Results

We evaluated the different approaches for schedulability along with the computation times under different configuration requirements. For each configuration, we generated ten different tasksets and for each taskset, we simulated 1000 requests for utilization modifications. For each utilization request, we assigned the new utilization values chosen from a uniform distribution between the desired utilization and the maximum utilization. Moreover, each task requesting increased utilization was chosen randomly.

Schedulability. The schedulability of the different configurations for the three methods is shown in Fig. 2. Here, GP refers to the global re-partitioning approach of ORR et al. while PCM refers to the per-core utilization modification approach and CA refers to the combined approach of per-core modifications and re-partitioning. The results show that the combined approach has a 100 percent success value while the per-core modifications have a success value of 95 percent in the worst-case indicating that in most cases, the per-core modification is sufficient. In case of failure, the limited re-partitioning associated with the combined approach can find a schedulable solution. The variation in successful requests for certain configurations is mostly due to the randomness in the generated tasksets and the tasks requesting for the modified utilization.

For configurations with a reduced number of tasks (Table 6), the schedulability performance of the per-core approach was around 93 percent in the worst-case scenario, which was relatively worse than compared to tasksets with a larger number of tasks while the other two approaches had 100 percent success (See Fig. 5).

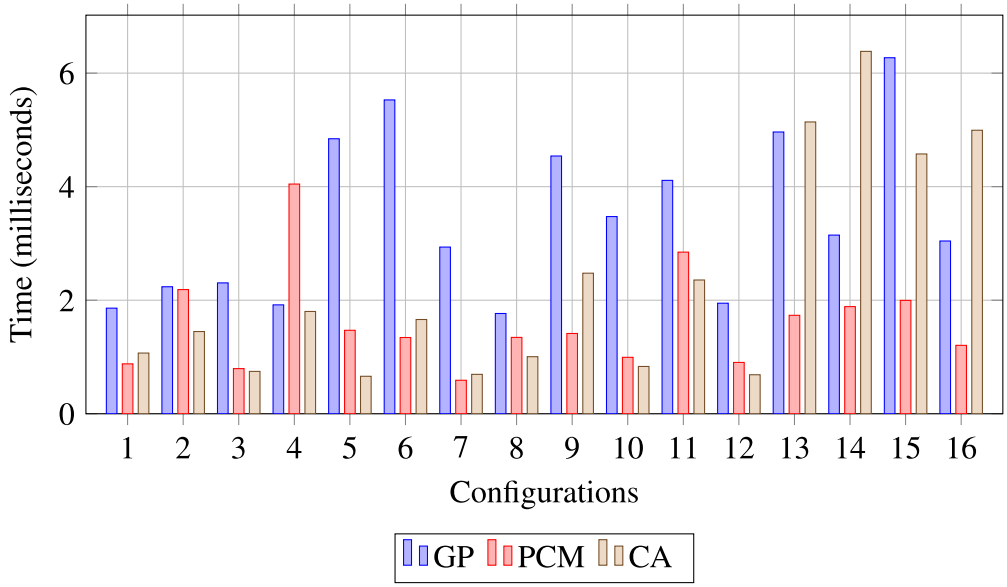


Fig. 4. Worst case computation times of the different approaches.

Table 6
Configuration with a smaller number of tasks.

Configuration	Maxmin	No. of tasks	No. of processors
1	1, 0.5	20	2
2	1, 0.5	20	4
3	1, 0.5	20	8
4	0.5, 0.5	20	2
5	0.5, 0.5	20	4
6	0.5, 0.5	20	8

Computation time. For each configuration, we measured the computation time required to find a schedulable solution and to report a failure if no solution is found through a prototype implementation of the proposed approaches on a Windows system. Fig. 3 shows the average computation time per request for each of the different configurations. The average was calculated by measuring 10000 requests. The results indicate that the per-core and the combined approach perform relatively better compared to the global re-partitioning approach. In certain scenarios, the average computation time of the combined approach is 6 times better than the global re-partitioning approach. This improvement is mostly due to the fact that most requests are managed within the core and that re-partitioning is done only if necessary, unlike the global re-partitioning approach where every new request results in re-partitioning.

Fig. 4 shows the worst-case computation times taken by the different approaches. The results indicate that for most configurations, the proposed approaches perform better than the global partitioning approach. In certain cases, however, the worst-case performance of the combined approach is similar (or worse in limited scenarios) to the global re-partitioning approach.

For configurations with a reduced number of tasks (Table 6), the computation performance of all the approaches was better compared to the configurations with a larger number of tasksets. Comparing the performance of different approaches for the same number of tasks (See Fig. 6), the average value of the combined approach was up to two times better than the global partitioning approach. The worst-case performance of the combined approach (See Fig. 7) was worse than the global approach for most of the evaluated configurations.

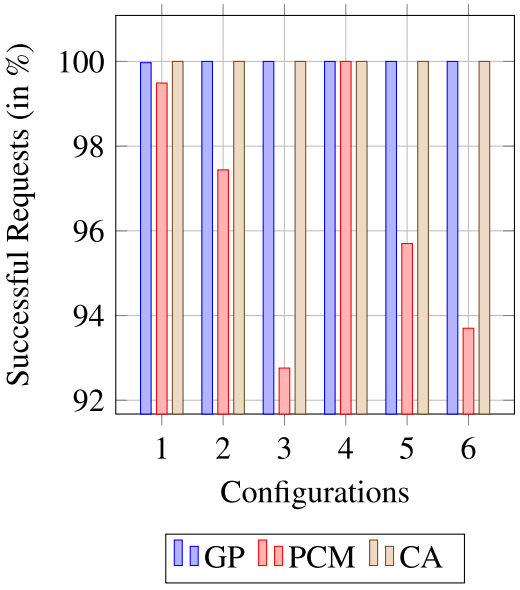


Fig. 5. Schedulability performance of smaller tasksets.

5. Related work

Hierarchical scheduling of periodic and sporadic real-time applications was encapsulated in a compositional real-time scheduling framework by Lee et al. [3]. In this framework, the computational demand of an application was abstracted with a single demand interface as a pair of capacity and period and the resource supply server was abstracted as a periodic resource model where each server was guaranteed a reserved capacity Θ every Π time units. Easwaran et al. [8] extended the periodic resource model to include the deadline parameter, where each server was guaranteed a reserved capacity Θ within D time units, in every time interval Π. Dewan et al. [9,10] provided algorithms to find an approximate allocation of bandwidth for a set of periodic and sporadic tasks under the periodic resource model. The periodic resource model was further extended for the case of multiprocessors by Shin et al. [11] where the periodic resource supply model was augmented with a parameter indicating the maximum concurrency

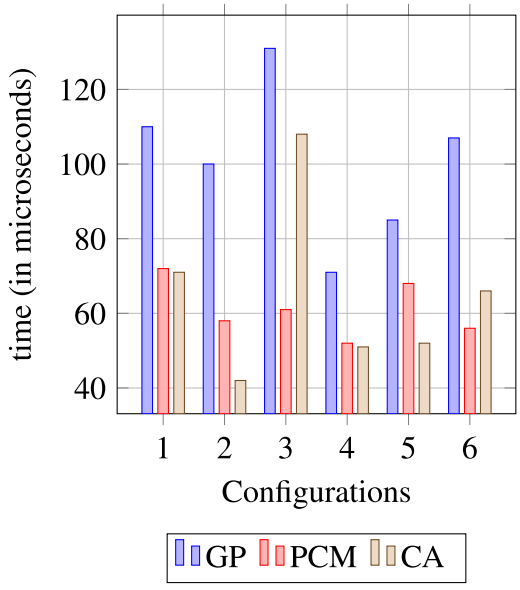


Fig. 6. Average computation times for smaller tasksets.

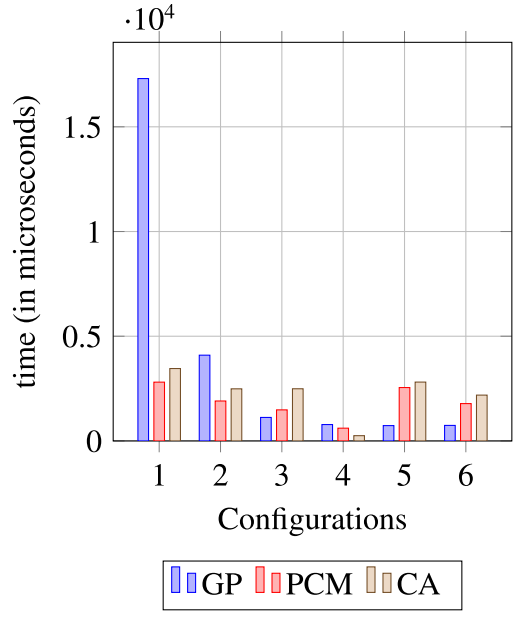


Fig. 7. Worst case computation times for smaller tasksets.

with which the resource supply is provided. The tasks assigned to this resource supply were then scheduled using a global scheduler. Bini et al. [12,13] provide a more expressive and general model for specifying multiprocessor resource supply in the form of the parallel supply function. Lee et al. [14] build upon the multiprocessor resource supply model and provide a cache-aware compositional analysis for the minimum parallelism resource supply form for the global EDF scheduling policy.

The concept of elastic tasks was introduced by Buttazzo et al. [15] to model applications whose computational demands can occasionally exceed the available capacity by allowing the application to modify the demand by changing the frequency of its jobs through an elastic coefficient. This was extended to address resource sharing within the elastic task model in [4]. Guangming [16] provided an earlier time for accelerating and adding tasks for the elastic scheduling approach. Chantem et al. [17,18] reformulated the problem as a quadratic optimization problem and showed that the original elastic tasks compression algorithm was indeed a solution to solving a quadratic problem. Tian et al. [19] extended the modified problem to include a "Quality-of-Control" metric as a part of the objective function of the quadratic optimization problem. More recently, Orr et al. [20,6] provided algorithms to schedule sequential elastic tasks on multiprocessor systems and further extended the concept of the elastic task to federated DAG-based parallel task systems in [21,22]. Beccari et al. [23,24] provided alternative algorithms to schedule similar applications by expressing the task period ranges in a linear programming formulation.

Another commonly used approach to schedule application tasks with timing variability is to modify the resource reservations. Thiele et al. [25] provide an online reconfiguration algorithm for the constant bandwidth server. Khalilzad et al. [26] proposed an adaptive hierarchical scheduling model to accommodate the adaptive behaviour of the periodic and sporadic tasks by changing the bandwidth allocation. In contrast, this paper assumes that a bandwidth allocated for a server under the periodic resource model remains constant and that the workload within the server can be adapted according to the elastic task model. However, if the elastic assignment fails, a request for a new bandwidth allocation will be made. We note that the proposed solution does not take into account possible bandwidth reclamation or mixed-criticality-based approaches to assign new bandwidths if no schedulable allocation can be made. Instead, we leave it to the individual application to handle such failures. For uniprocessor systems, we provide a method to schedule elastic applications within a periodic resource supply model reservation in [27].

6. Conclusion

Scheduling of elastic applications on reservation-based multiprocessors systems has not been extensively studied. To address this, we proposed a scheduling framework based on the minimum-parallelism resource reservation model and compared different partition-based EDF scheduling mechanisms. The proposed methods introduce a relatively intuitive approach to scheduling elastic tasks under reservation schemes on multiprocessors combining the advantages of the simplicity offered by the MPS reservation and flexibility of the elastic tasks. The evaluation results indicate the proposed methods outperform the current state-of-art approaches on average, while in the worst case, none of the approaches outperforms the other. In future work, we intend to investigate methods to improve the worst-case performance and evaluate the results on a real system.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

The research leading to these results has received funding from the European Union's Horizon 2020 research and innovation programme under the Marie Sklodowska Curie grant agreement No. 764785, FORA—Fog Computing for Robotics and Industrial Automation, and by the Swedish Knowledge Foundation (KKS) under the projects FIESTA, HERO and DPAC.

References

[1] H. Leontyev, J.H. Anderson, A hierarchical multiprocessor bandwidth reservation scheme with timing guarantees, in: Euromicro Conference on Real-Time Systems, ECRTS, 2008, pp. 191–200, http://dx.doi.org/10.1109/ECRTS.2008.22.

[2] K. Yang, J.H. Anderson, On the dominance of minimum-parallelism multiprocessor supply, in: IEEE Real-Time Systems Symposium, RTSS, IEEE, 2016, pp. 215–226, http://dx.doi.org/10.1109/RTSS.2016.029.

[3] I. Shin, I. Lee, Compositional real-time scheduling framework with periodic model, ACM Trans. Embed. Comput. Syst. 7 (3) (2008) 30:1–30:39, http://dx.doi.org/10.1145/1347375.1347383.

[4] G.C. Buttazzo, G. Lipari, M. Caccamo, L. Abeni, Elastic scheduling for flexible workload management, IEEE Trans. Comput. 51 (3) (2002) 289–302, http://dx.doi.org/10.1109/12.990127.

[5] J.M. López, J.L. Díaz, D.F. García, Utilization bounds for EDF scheduling on real-time multiprocessor systems, Real-Time Syst. 28 (1) (2004) 39–68, http://dx.doi.org/10.1023/B:TIME.0000033378.56741.14.

[6] J. Orr, S. Baruah, Algorithms for implementing elastic tasks on multiprocessor platforms: A comparative evaluation, Real-Time Syst. 57 (2021) 227–264, http://dx.doi.org/10.1007/s11241-020-09358-9.

[7] D. Griffin, I. Bate, R.I. Davis, Generating utilization vectors for the systematic evaluation of schedulability tests, in: IEEE Real-Time Systems Symposium, RTSS, 2020, pp. 76–88, http://dx.doi.org/10.1109/RTSS49844.2020.00018.

[8] A. Easwaran, M. Anand, I. Lee, Compositional analysis framework using EDP resource models, in: IEEE Real-Time Systems Symposium, RTSS, IEEE Computer Society, 2007, pp. 129–138, http://dx.doi.org/10.1109/RTSS.2007.36.

[9] N. Fisher, F. Dewan, A bandwidth allocation scheme for compositional real-time systems with periodic resources, Real-Time Syst. 48 (3) (2012) 223–263, http://dx.doi.org/10.1007/s11241-011-9144-7.

[10] F. Dewan, N. Fisher, Bandwidth allocation for fixed-priority-scheduled compositional real-time systems, ACM Trans. Embed. Comput. Syst. 13 (4) (2014) 91:1–91:29, http://dx.doi.org/10.1145/2560038.

[11] I. Shin, A. Easwaran, I. Lee, Hierarchical scheduling framework for virtual clustering of multiprocessors, in: Euromicro Conference on Real-Time Systems, ECRTS, 2008, pp. 181–190, http://dx.doi.org/10.1109/ECRTS.2008.28.

[12] E. Bini, M. Bertogna, S. Baruah, Virtual multiprocessor platforms: Specification and use, in: IEEE Real-Time Systems Symposium, RTSS, 2009, pp. 437–446, http://dx.doi.org/10.1109/RTSS.2009.35.

[13] A. Burmyakov, E. Bini, E. Tovar, Compositional multiprocessor scheduling: The GMPR interface, Real-Time Syst. 50 (3) (2014) 342–376, http://dx.doi.org/10.1007/s11241-013-9199-8.

[14] M. Xu, L.T. Phan, I. Lee, O. Sokolsky, S. Xi, C. Lu, C. Gill, Cache-aware compositional analysis of real-time multicore virtualization platforms, in: IEEE Real-Time Systems Symposium, RTSS, 2013, pp. 1–10, http://dx.doi.org/10.1109/RTSS.2013.9.

[15] G.C. Buttazzo, G. Lipari, L. Abeni, Elastic task model for adaptive rate control, in: IEEE Real-Time Systems Symposium, RTSS, IEEE, 1998, pp. 286–295, http://dx.doi.org/10.1109/real.1998.739754.

[16] Q. Guangming, An earlier time for inserting and/or accelerating tasks, Real-Time Syst. 41 (3) (2009) 181–194, http://dx.doi.org/10.1007/s11241-009-9066-9.

[17] T. Chantem, X.S. Hu, M.D. Lemmon, Generalized elastic scheduling, in: IEEE Real-Time Systems Symposium, RTSS, 2006, pp. 236–245, http://dx.doi.org/10.1109/RTSS.2006.24.

[18] T. Chantem, X.S. Hu, M.D. Lemmon, Generalized elastic scheduling for real-time tasks, IEEE Trans. Comput. 58 (4) (2009) 480–495, http://dx.doi.org/10.1109/TC.2008.175.

[19] Y.C. Tian, L. Gui, QoC elastic scheduling for real-time control systems, Real-Time Syst. 47 (6) (2011) 534–561, http://dx.doi.org/10.1007/s11241-011-9133-x.

[20] J. Orr, C. Gill, K. Agrawal, J. Li, S. Baruah, Elastic scheduling for parallel real-time systems, Leibniz Trans. Embed. Syst. 6 (1) (2019) 5:1–5:14, http://dx.doi.org/10.4230/LITES-v006-i001-a005.

[21] J. Orr, C.D. Gill, K. Agrawal, S.K. Baruah, C. Cianfarani, P. Ang, C. Wong, Elasticity of workloads and periods of parallel real-time tasks, in: International Conference on Real-Time Networks and Systems, RTNS, ACM, 2018, pp. 61–71, http://dx.doi.org/10.1145/3273905.3273915.

[22] J. Orr, J.C. Uribe, C.D. Gill, S.K. Baruah, K. Agrawal, S. Dyke, A. Prakash, I. Bate, C. Wong, S. Adhikari, Elastic scheduling of parallel real-time tasks with discrete utilizations, in: International Conference on Real Time Networks and Systems, RTNS, ACM, 2020, pp. 117–127, http://dx.doi.org/10.1145/3394810.3394824.

[23] G. Beccari, S. Caselli, M. Reggiani, F. Zanichelli, Rate modulation of soft real-time tasks in autonomous robot control systems, in: Euromicro Conference on Real-Time Systems, ECRTS, IEEE Computer Society, 1999, pp. 21–28, http://dx.doi.org/10.1109/EMRTS.1999.777446.

[24] G. Beccari, S. Caselli, F. Zanichelli, A technique for adaptive scheduling of soft real-time tasks, Real-Time Syst. 30 (3) (2005) 187–215, http://dx.doi.org/10.1007/s11241-005-2461-y.

[25] P. Kumar, N. Stoimenov, L. Thiele, An algorithm for online reconfiguration of resource reservations for hard real-time systems, in: Euromicro Conference on Real-Time Systems, ECRTS, IEEE, 2012, pp. 245–254, http://dx.doi.org/10.1109/ECRTS.2012.28.

[26] N.M. Khalilzad, T. Nolte, M. Behnam, M. Åsberg, Towards adaptive hierarchical scheduling of real-time systems, in: Euromicro Conference on Real-Time Systems, ECRTS, 2011, pp. 1–8, http://dx.doi.org/10.1109/ETFA.2011.6059019.

[27] M.S. Shaik, S. Mubeen, F. Marković, A.V. Papadopoulos, T. Nolte, Scheduling elastic applications in compositional real-time systems, in: IEEE International Conference on Emerging Technologies and Factory Automation, ETFA, 2021.